

Fulfillment Center (3) · Project Report

Operating Systems Course · Group 23

Antonio Salvaterra · Giovanni Zalunardo · Alvise Zamuner

Abstract

The system emulates an e-commerce fulfillment center as cooperating processes: a multi-threaded warehouse (the core), N supplier processes, and three Bash scripts (`bootstrap.sh`, `order.sh`, `manage.sh`) with two small C helpers (`order_helper`, `manage_restock_helper`) for the binary IPC. All message passing uses FIFOs; signals are used only for control operations such as status requests and shutdown. This report documents the design decisions file by file.

1 System architecture

The system is split into processes, each with one responsibility (Figure 1). The warehouse is a single multi-threaded process that owns the in-memory inventory and runs the order pipeline through three thread pools (Receiver, Picker, Packer) plus one Restock thread. The suppliers are separate C processes. The Bash scripts drive the system and validate user input, but the binary IPC itself is delegated to two C helpers, because Bash cannot reliably put the fixed-size binary records the protocol uses onto a FIFO. This keeps input validation (Bash) cleanly separated from binary transport (C).

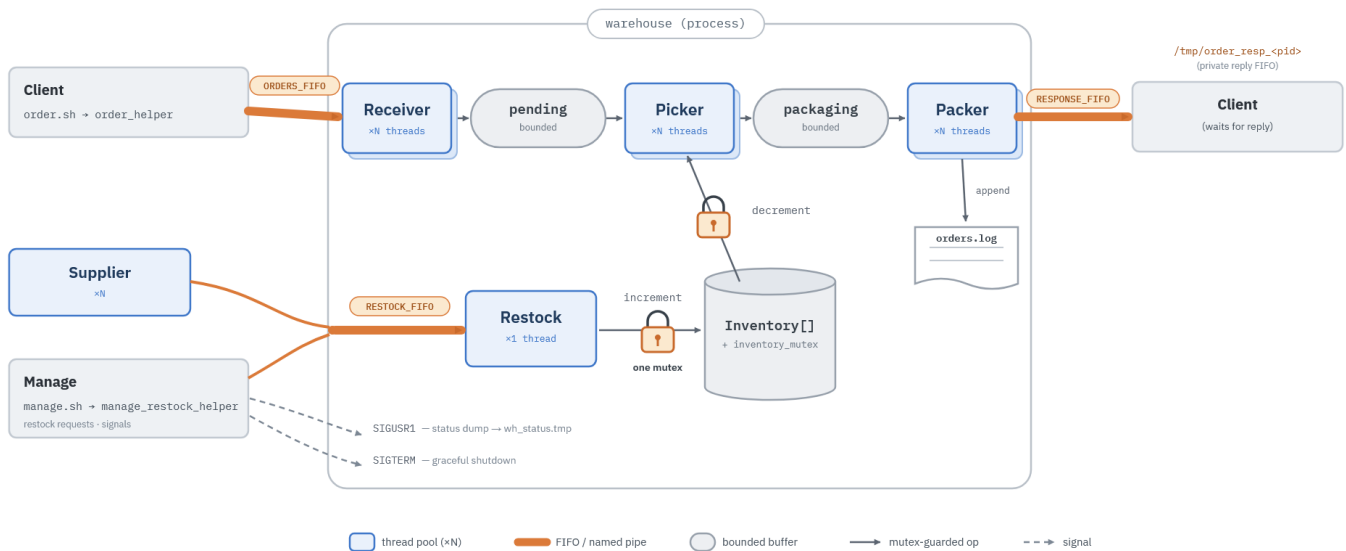


Figure 1: IPC architecture. Thread pools (blue) exchange orders through bounded buffers; the orange paths are FIFOs; the Inventory is one shared array updated under a single mutex by Pickers and Restock.

1.1 Message format and atomic FIFO writes

The three messages exchanged over the FIFOs are fixed-size binary C structs: `OrderRequest`, `OrderResponse` and `RestockMsg`. Each is sent with a single `write()` of `sizeof(struct)`. Two properties follow from this. First, the structs hold no pointers, which would be meaningless in another address space, so every string is a fixed-size array. Second, each struct is well below `PIPE_BUF` (4096 B on Linux), so each FIFO write is atomic (`man 7 pipe`): concurrent writers, the many clients on `ORDERS_FIFO` and the many suppliers on `RESTOCK_FIFO`, never interleave, and a reader always gets a whole message. We therefore only need one mutex on the read side, and each thread reads exactly one whole message: we do not have to add length prefixes or separators to mark where one message ends and the next begins. This is what keeps the IPC consistent.

1.2 Low-level I/O and error handling

We use raw file descriptors (`open/read/write/close`) instead of stdio `FILE*` streams, because descriptors give direct control over byte counts, atomicity and errors, which is what binary IPC needs. A `read` or `write` can be partial or interrupted by a signal (`EINTR`), so the helpers `read_all/write_all` loop until the whole struct has been transferred, and every return value is checked. One signal needs care: when a peer closes a FIFO, the next `write` would raise `SIGPIPE`, whose default action is fatal. We set `SIGPIPE` to `SIG_IGN` in every C program, so the `write` instead fails with `EPIPE` and is handled as a normal error, `ERR_WAREHOUSE_DOWN` in the clients. `SIGPIPE` is raised synchronously by the

calling `write`, so ignoring it loses no information; and since `SIG_IGN` is a process-wide disposition, the multi-threaded warehouse needs no per-thread signal handling for it.

1.3 Error codes

Every operation returns an `ERR_*` code (Table 1), defined once in `common.h` and mirrored with the same value in the Bash scripts, so an outcome crosses the C/Bash boundary through `$?`. The values are small integers (0 to 8) so they double as process and script exit codes.

Code	Value	Meaning	Code	Value	Meaning
<code>ERR_OK</code>	0	success	<code>ERR_PARTIAL_FILL</code>	5	partial shipment
<code>ERR_ITEM_NOT_FOUND</code>	1	item not found	<code>ERR_WAREHOUSE_DOWN</code>	6	warehouse down
<code>ERR_OUT_OF_STOCK</code>	2	stock exhausted	<code>ERR_TIMEOUT</code>	7	reply timed out
<code>ERR_INVALID_QTY</code>	3	non-positive qty	<code>ERR_USAGE</code>	8	wrong arguments
<code>ERR_IO</code>	4	I/O error			

Table 1: error codes.

2 bootstrap.sh: startup

`bootstrap.sh` sets up and launches the system, then exits leaving it running. It validates the arguments and `inventory.csv`, checks the executables exist, cleans up any previous run, creates the FIFOs, generates the supplier configuration files, and starts the warehouse and suppliers in the background, saving their PIDs.

2.1 Supplier configuration (round-robin)

Every item must be served by at least one supplier. We iterate over the inventory and assign each item to the next supplier in turn, wrapping back to the first after the last: a plain round-robin that guarantees full coverage and an even split. When there are more items than suppliers, the wrap-around gives each supplier several items. When there are more suppliers than items, the surplus suppliers would otherwise be idle, so each of them is assigned one random item as redundant capacity. All assignments use a fixed quantity per shipment; primary assignments use a random interval between 5 and 15 seconds, while redundant assignments use a longer 16 to 30 s interval, so backup shipments are spread out and do not congest the FIFO.

2.2 All-or-nothing startup

There are two distinct cleanup moments. At startup we first remove anything a previous run may have left (FIFOs, PID files, configs, old log), so we always begin from a clean slate. During the launch itself we treat startup as a transaction: an `EXIT` trap plus two flags (`RUNTIME_CREATED`, `BOOTSTRAP_SUCCESS`) roll everything back if any step fails, killing whatever was already started and removing what was created, so an aborted launch never leaves half a system or orphan processes behind. On success the traps are disarmed so the background processes survive the script. We also trap `INT` and `TERM` so that a signal becomes a controlled exit that still runs the cleanup. After launching the warehouse we confirm it is alive with `kill -0` before starting the suppliers.

3 common.h / common.c: the shared interface

`common.h` fixes the binary layout that every process includes, so all programs agree on the same binary format. It holds only what crosses process boundaries: FIFO and file paths, error codes, string-size limits, and the three message structs. `OrderRequest` also carries `resp_fifo`, the path of the client’s private reply FIFO, so that path travels with the order down the pipeline and whichever thread answers reads it straight from the live `Order`. The warehouse-internal types (`Inventory`, `BoundedQueue`, `Order`) stay in `warehouse.c`, since they never leave the process. `common.c` centralizes the helpers used by more than one program, in one place to avoid duplication:

- `setup_handler`: installs a handler with `sigaction` and `sa_flags = 0` (no `SA_RESTART`), so a slow syscall (e.g. a `write()`) can be interrupted rather than restarted;
- `write_all`: loop until the whole struct has been written, retrying on `EINTR`;
- `read_line_from_fd`: reads one line at a time from a descriptor, used for the CSV and `.conf` files;
- `open_fifo_r_dw`: opens a FIFO ready to be read (below).

3.1 Opening a read FIFO

In our design every process that reads a FIFO needs the same setup, which is why `open_fifo_r_dw` lives in `common.c`: the reader must hold the FIFO open before any writer arrives, and then block on `read` without busy-waiting. The warehouse uses it for its two FIFOs, and `order_helper` for its private reply FIFO. The setup is three steps:

1. `open(path, O_RDONLY | O_NONBLOCK)`; a plain `O_RDONLY` would block until some writer opens the FIFO. We use `O_NONBLOCK` only to open the read end first, before the dummy write end exists.
2. `open(path, O_WRONLY)`; this dummy write end is kept open by the reader process itself. Its purpose is to prevent accidental EOF: if all external writers temporarily disconnect, the read side should keep blocking for future messages, not terminate as if the FIFO were permanently closed.

3. `fcntl(rfd, F_SETFL, ... & ~O_NONBLOCK)`; clears `O_NONBLOCK` so the following reads block normally and the thread sleeps instead of spinning.

POSIX does not specify the behaviour of a single `O_RDWR` open on a FIFO, so to stay portable we keep the read-end plus write-end pair.

4 warehouse.c: the multi-threaded core

Four thread types form the pipeline. Receivers read an `OrderRequest`, validate it, and place the order into the `pending` queue. Pickers take from `pending`, sleep 1 to 3 s (simulated picking), decrement stock and move the order to `packaging`. Packers take from `packaging`, sleep 1 to 3 s, send the reply and append to the log. A separate Restock thread consumes `RESTOCK_FIFO` and increments stock. An order lives as the internal `Order` struct: the request, an internal `order_id` we assign as an incrementing counter for logging and tracing, the shipped and rejected counts, the error code and the state. The inventory, queues, descriptors and mutexes are created in `main` and passed to the threads by reference inside per-role argument structs, so each thread sees only what it uses; the only globals are the signal flags, which the handlers set.

4.1 Order lifecycle

An order moves `RECEIVED` to `PICKING` to `PACKING` to `SHIPPED`, or to `REJECTED` (Figure 2). Validation happens at two points. The Receiver rejects input errors at once: a non-positive quantity, an unknown item, or zero stock. The binding decision on stock is the Picker's, taken under the inventory lock: it ships `min(stock, requested)`, so a full match is `SHIPPED`, a short fill is `PARTIAL`, and nothing left is `REJECTED`. The reply is not always sent by the same thread: whichever stage settles the order answers the client. A Receiver that rejects bad input replies itself; a Picker that finds the item gone replies itself; a Packer replies on success or partial. All of them fill the same `OrderResponse` struct, defined for exactly this purpose, and write it to the client's reply FIFO with a non-blocking open, so a client that has died fails fast (`ENXIO`) instead of blocking the thread. `orders.log` is opened `O_APPEND` and is pipe-separated, so `manage.sh` can parse it with standard text tools.

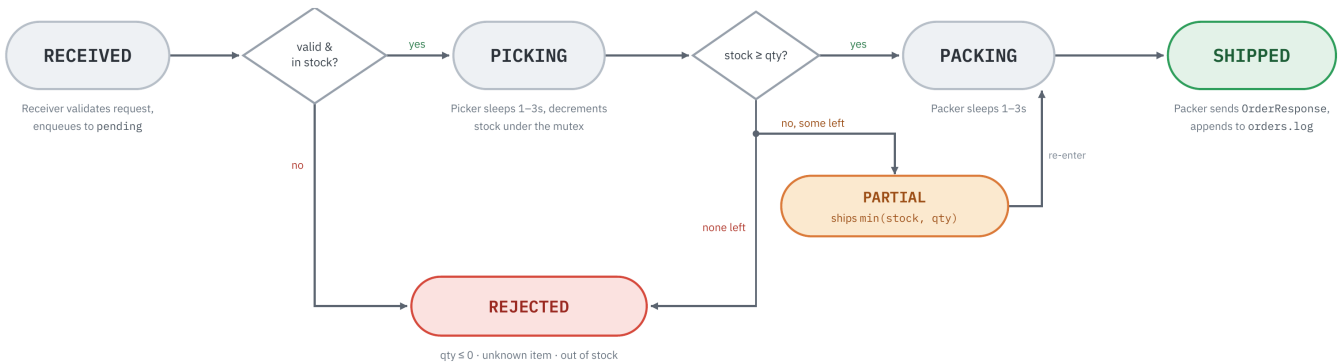


Figure 2: order states, with the rejection and partial-fulfilment paths.

4.2 Synchronization

The two queues are circular bounded buffers, each guarded by a mutex and two condition variables (`not_full`, `not_empty`); a thread that has to wait re-checks its condition in a loop around the wait, so a spurious wake-up can never let it proceed too early, and blocked threads sleep in the kernel and never spin. We use a mutex with condition variables rather than counting semaphores because the wait condition is explicit and the shutdown flag fits into the same condition, so raising it releases every waiting thread; a full buffer is therefore only a transient state and never an error, which is why there is no `QUEUE_FULL` code. The inventory is a single array under one mutex shared by Pickers (decrement) and Restock (increment). One mutex is sufficient because the critical section is just a lookup and an update, which is tiny next to the 1 to 3 s sleeps that run outside the lock, so contention is negligible and a per-item or reader-writer lock would only add complexity. That single lock also removes the race condition on the inventory by construction, including two clients taking the last unit at the same moment.

4.3 Signals

Signals are blocked with `sigprocmask` before the threads are created, so the mask is inherited and only `main` receives them; a handler does the single async-signal-safe action, setting a flag. We do it this way on purpose to avoid a classic deadlock: a handler must not lock a mutex, because if the signal arrived while a thread already held that mutex the handler would block forever. So the `SIGUSR1` status dump runs in `main`, not in the handler: `main` copies the inventory under the lock, writes it outside the lock to a temporary file, then `rename(2)`s it onto the status path. The rename is atomic, so `manage.sh` reads the complete new file, never a half-written dump. `main` waits for signals with `sigsuspend`, which unblocks and sleeps in one atomic step.

4.4 Graceful shutdown

On `SIGTERM` with orders still in flight we drain the pipeline in order so nothing already accepted is lost (Figure 3). The warehouse keeps its own write end open on each `FIFO` (the trick from the opening pattern), while the suppliers hold their own write ends on `RESTOCK_FIFO`. To stop the Restock thread we send a stop sentinel (a `RestockMsg` with `supplier_id = -1`) and close our write end: the thread exits as soon as it reads the sentinel, which is what lets it stop even while a long-lived supplier still holds a write end and EOF has not arrived. Then we join Receivers and Restock (no new work reaches `pending`), raise shutdown on `pending` and join the Pickers (they drain it first), and raise shutdown on `packaging` and join the Packers. The order matters: shutting `pending` down before the Receivers are joined could let a Picker exit while a Receiver is still enqueueing, orphaning an order and hanging its client. Raising the shutdown flag wakes the waiting threads with a cascading `pthread_cond_signal`, where each thread that leaves signals the next. Finally, the main thread cleans up the remaining resources by closing the file descriptors, unlinking the `FIFOs` and the status file, and destroying the mutexes and condition variables.



Figure 3: ordered shutdown on `SIGTERM`; each stage is drained before the next is closed.

5 supplier.c: periodic restocking

Each supplier reads its `.conf` and sends one `RestockMsg` per item on `RESTOCK_FIFO`, each item on its own interval. It opens the `FIFO O_WRONLY` (blocking until the warehouse's read end exists; `bootstrap.sh` starts the warehouse first), retrying on `EINTR`. The writes are small fixed-size structs, so concurrent suppliers never interleave.

5.1 Scheduling with relative countdowns

We schedule deliveries with per-item countdowns rather than absolute timestamps so the supplier only needs to sleep until the next shipment is due. On each loop it takes the smallest remaining countdown, sleeps once for that amount of time, then subtracts the time actually slept from all countdowns, ships the items that reached zero, and rearms each of them to its own interval. This keeps the process blocked in the kernel between deliveries instead of polling, and makes termination prompt: if `SIGTERM` interrupts `sleep`, the supplier wakes at once and exits instead of waiting out the interval. Since countdowns are updated from the time actually slept, delayed or interrupted sleeps do not create a delivery backlog. At process level, suppliers remain independent: if one crashes, it simply stops writing, while the warehouse and the remaining suppliers continue running because no thread waits for a specific supplier.

6 order.sh and order_helper

`order.sh` validates the arguments (it restricts `client_id` to letters, digits and underscores so the pipe-separated log stays clean, and requires `item_id` and `quantity` to be at least 1), checks the warehouse is alive and that `ORDERS_FIFO` exists, then delegates the binary IPC to `order_helper` and returns its exit code. An `echo > fifo` would not work: `OrderRequest` is a fixed-size binary record and needs a `write(2)` of `sizeof`.

`order_helper` runs the request/reply protocol, and its delicate point is the private reply `FIFO`. Because the warehouse answers with a non-blocking open, the client must already hold the read end of its own reply `FIFO` (whose path it shipped inside `OrderRequest`) before sending the request, or the reply is lost; it sets this up with the same open pattern. A `SIGALRM` timeout (30 s, handler without `SA_RESTART`) covers the whole open/write/read chain: if the warehouse dies after accepting the order, the read on the private `FIFO` would never see EOF (the dummy write end keeps it open), so the alarm interrupts it and we return `ERR_TIMEOUT`. Validation is layered: `order.sh` (user input), `order_helper` (record format), warehouse (the final decision on existence, quantity and stock).

7 manage.sh and manage_restock_helper

`manage.sh` controls the running system over signals and IPC. `status` removes the old state file, sends `SIGUSR1`, waits for the status file the warehouse publishes with an atomic `rename(2)`, and prints processes, queues and inventory. `report` reads `orders.log` and summarizes it with `grep`, `cut`, `sort`, `uniq`, and `wc`: total, shipped, partial and rejected orders, units shipped, and the most-ordered items. The top-5 ranks items by number of orders, not by units shipped. `shutdown` sends `SIGTERM` to the suppliers first (so no new restocks arrive while the warehouse drains), then to the warehouse, waits up to about 20 s for it to exit, resorting to `SIGKILL` only if it does not, before cleaning up the IPC resources.

`restock` delegates the IPC to `manage_restock_helper`, which writes a `RestockMsg` with `supplier_id = 0` (the

manual marker), processed by the same Restock thread that serves the suppliers. We chose a FIFO over a signal such as `SIGUSR2` for three reasons: a signal carries no data (we would still need a file for the item and quantity); a signal handler could not safely apply the restock, because `pthread_mutex_lock` is not async-signal-safe and could deadlock against a Picker holding the lock (the same reason the status dump runs in `main`); and signals can be merged or lost, while the FIFO queues every message. The helper opens the FIFO non-blocking, so a missing warehouse gives `ENXIO`, reported as `ERR_WAREHOUSE_DOWN`, instead of blocking.

8 Edge cases, build and interfaces

The required edge cases are handled by construction: the inventory mutex serializes the last-unit race and a concurrent restock; the condition variables handle a full or empty queue without spinning; the Receiver rejects an unknown item or a non-positive quantity at once; `SIGTERM` drains the pipeline before exit; and a crashing supplier blocks nothing because suppliers are independent writers and the warehouse never waits for a specific supplier. All critical I/O and IPC calls are checked, and user-visible failures are propagated through `ERR_*` codes.

The Makefile builds the four executables (`build`), removes the binaries and the IPC resources (`clean`), and runs a scenario through `bootstrap.sh` via the `ARGS` variable (`run`); `make help` prints the usage. Every object depends on `common.h`, so a change to the interface recompiles everything. The five required entry points keep their exact signatures:

```
./bootstrap.sh <nr> <np> <n timer> <cap> <nsup> <csv>
./warehouse    <nr> <np> <n timer> <cap> <csv>
./supplier      <id> <config_file>
./order.sh      <client_id> <item_id> <qty>
./manage.sh     <status | restock <id> <qty> | report | shutdown>
```